Model Prediction to Housing Sale Price in Queens, NY

Ye Htut Maung

May 2025

Final project for Math 342W Data Science at Queens College

By [Ye Htut Maung]
In collaboration with:
[Sergio E. Garcia Tapia]
[Brian Rojas Hernandez]

Abstract

In this project, we developed and compared multiple predictive models to estimate apartment sale prices in Queens using a cleaned dataset from 2016–2017. We explored linear regression, regression trees, and random forests, with a focus on data preprocessing, imputation using missForest, and honest model evaluation via a 3-way data split. Our findings show that the linear model outperformed tree-based models in out-of-sample performance, likely due to the underlying linear structure in the data and limited training size. While the model is not yet production-ready, it offers insights into how data handling and modeling choices impact predictive performance.

pagebreak

1 Introduction

In this project, we are going to develop a predictive model to estimate the sale price of apartments in Queens, New York. A predictive model means that we are going to provide some features that is related or contribute to the response value. Then, the model will predict the sale price based on the provided features. We are going to use real estate data collected between February, 2016 and February, 2017. The dataset was harvested with Amazon's MTurk and it is the raw data representation found at MLSI. It includes housing units that are either condominiums or cooperative apartments, with sale prices capped at $1 million. Our goal of this project is to produce a reliable model that can predict sale prices for new apartment listings based on features provided in the dataset.

The unit of observation is an individual apartment listing in Queens. The response variable is sale_price, which is the final selling price of the apartment. The dataset includes various features from the number of rooms, type of apartment, pet policy to community district and listing price.

To build a predictive model, we implemented three different approaches: a regression tree model, a linear regression model, and a random forest model. Each model was trained on a subset of the data with imputed missing values. After building models, we validate with out-of-sample metrics to access our predictive performance. Throughout the process, we explored feature transformations, data cleaning and reports of performance metrics.

2 The Data

The dataset used in this project comes from the "housing_data_2016_2017.csv" file provided in the Math 342W GitHub. It was originally obtained via Amazon Mechanical Turk from the Multiple Listing Service (MLSI). The original dataset contains 2,230 rows and 55 columns. In the following steps, we will perform featurization and handle errors and missingness. The population of interest is the set of apartments sold in mainland Queens under $1 million during the specified time period. There are some columns that are not related to housing price, and we will drop those. We also identified some outliers, including in the zip_code column. For example, some zip codes appear only once or twice in the dataset. If these appear only in the test set and not in the training set, the model cannot make valid predictions for them. This would lead to errors or extrapolation, which falls outside the range of the training data. We will discuss each feature in more detail in the following subsection.

2.1 Featurization

The original dataset includes 55 columns, but not all of them are relevant to predicting sale price. Therefore, we dropped the following columns: "HITId", "HITTypeId", "Title", "Description", "Reward", "CreationTime", "RequesterAnnotation", "Expiration", "AssignmentId", "WorkerId", "AssignmentStatus", "AcceptTime", "SubmitTime", "AutoApprovalTime", "ApprovalTime", "LifetimeApprovalRate", "Last30DaysApprovalRate", "Last7DaysApprovalRate", "Keywords", "NumberOfSimilarHITs", "LifetimeInSeconds", "RejectionTime", "RequesterFeedback", "MaxAssignments", "AssignmentDurationInSeconds", "AutoApprovalDelayInSeconds", "WorkTimeInSeconds", "model_type", "date_of_sale". Most of these are technical metadata fields collected from Amazon Mechanical Turk and are unrelated to housing price prediction.

The reason for dropping "model_type" is that it contains a large number of unique values, many of which appear to be typos or inconsistently formatted entries. While it is possible to manually clean and combine these values into fewer, more meaningful categories (likely fewer than 20), doing so would introduce subjective bias. Because the data in this column lacks consistency and is difficult to interpret reliably, we concluded that it would not meaningfully contribute to modeling the sale price and thus chose to exclude it. "date_of_sale" is also dropped as there are a lot of missiningness in this feature and so, the date of the sale cannot be imputed.

After cleaning and dropping irrelevant columns, we retained 23 features. These include both raw measurements and features that we created to better represent the observations. For some features, we cobmine and make a new column to represent more clear. We will look into each feature separately. Let's take a look of categorical features.

Feature Name	Description	Number of Missing-ness	Summary
cat_allowed	To identify whether cat is allowed in the apartment	0	yes: 37.13% no: 62.87%
coop_condo	Either coop or condo type	0	coop: 74.48% condo: 25.52%
dining_room_type	Whether combo, formal or other dining room type	448	combo: 53.70% formal: 34.79% other: 11.50%
dogs_allowed	To identify whether dog is allowed in the apartment	0	yes: 24.48% no: 75.52%
fuel_type	fuel_type of heating system	112	electric: 2.93% gas: 63.64% none: 0.14% oil: 31.35% other: 1.94%
garage_exists	Whether garage exists or not	0	yes: 18.12% no: 81.88%
kitchen_type	Type of kitchen: combo, eat_in, efficiency, none	16	combo: 18.02% eat_in: 42.55% efficiency: 38.35% none: 1.08%
community_district_num	The district number where the apartment is located	19	24: 8.59% 25: 27.86% 26: 16.1% 27: 5.02% 28: 26.37% 29: 3.08% 30: 10.22% other: 2.76%

Table 1: Placeholder caption for a 4-column, 20-row table with one heading.

Feature Name	Description	Number of Missingness	Summary
zip_code	The ZIP code where the apartment is located	0	11004: 1.57% 11005: 2.42% 11101: 0.31% 11102: 0.99% 11103: 0.36% 11104: 0.58% 11105: 0.67% 11106: 0.58% 11354: 6.33% 11355: 5.83% 11356: 1.08% 11357: 3.95% 11358: 0.90% 11360: 6.81% 11361: 0.90% 11362: 3.14% 11363: 0.49% 11364: 3.50% 11365: 0.99% 11367: 4.35% 11368: 2.64% 11369: 0.72% 11370: 0.27% 11372: 6.28% 11373: 3.50% 11374: 5.83% 11375: 13.45% 11377: 1.61% 11378: 0.22% 11379: 0.54% 11385: 0.67% 11414: 3.59% 11415: 4.44% 11417: 0.31% 11418: 0.18% 11421: 0.63% 11422: 0.31% 11423: 0.76% 11426: 0.90% 11427: 1.53% 11432: 3.36% 11435: 2.20% other: 0.31%

Now, take a look at numeric features.

Feature Name	Description	Number of Missing-ness	Summary
general_cost	This feature is about charges for maintenance fee for co op and common charge for condo	100	Range: [70, 4659] Average: 749.8 SD: 397.7021
parking_charge	Amount of money to be paid for parking	1671	Range: [6, 837] Average: 107.6 SD: 70.87827
total_taxes	Total taxes for the apartment	1646	Range: [11, 9300] Average: 2226 SD: 1850.094
listing_price_to_nearest_1000	Listing price of apartment to nearest 1000 (example, if it is 150, the listing price is $150,000	534	Range: [65, 1000] Average: 385.6 SD: 200.2584
approx_year_built	The approximate year when the apartment is built	40	Range: [1893, 2017] Average: 1963 SD: 21.07992
num_bedrooms	The number of bedrooms in an apartment	115	Range: [0, 6] Average: 1.65 (rounded 2) SD: 0.7436145
num_floors_in_building	The amount of floors in an apartment	650	Range: [1, 34] Average: 7.79 (rounded 8) SD: 7.5158

Feature Name	Description	Number of Missingness	Summary
num_full_bathrooms	The amount of full bathrooms which has both toilet and shower)	0	Range: [1, 3] Average: 1.23 (rounded 1) SD: 0.4446
num_half_bathrooms	The number of half bathrooms (toilet only, no shower)	2058	Range: [0, 2] Average: 0.95 (rounded 1) SD: 0.3023
num_total_rooms	The total number of rooms in the apartment	2	Range: [0, 14] Average: 4.14 (rounded 4) SD: 1.3459
pct_tax_deductibl	The percent of the maintenance cost that is tax deductible	1754	Range: [20, 75] Average: 45.4 SD: 6.9489
sq_footage	The total square footage of the apartment	1210	Range: [100, 6215] Average: 955.4 SD: 380.8647
walk_score	The walkability score of the apartment's location	0	Range: [7, 99] Average: 83.92 SD: 14.7473

2.2 Errors and Missingness

Several features in the dataset contained missing values or inconsistencies that needed to be addressed. For the common_charges and maintenance_cost columns, each applies to a different apartment type: common_charges is used for condos, while maintenance_cost applies to co-ops. Importantly, only one of these two values is present per row. To consolidate this information and handle the structured missingness, we created a new column called general_cost, which combines both sources. After this transformation, we dropped the original two columns. We also removed dollar signs ($) and commas to convert the string values into numeric format. The resulting general_cost column still had 100 missing values.

There are also many features that is string and have dollar signs or comma. We remove those characters and convert to numeric.

For the garage_exists variable, all missing values were assumed to mean "no garage" and were set accordingly. This assumption is reasonable given the binary nature of the variable and the likely tendency for garage presence to be explicitly recorded when available.

In the kitchen_type column, there was a single unusual entry labeled "1955", which likely represents a data entry error. We reclassified this entry as "none" to simplify and standardize the categories.

In the dining_room_type variable, the categories "dining area" and "none" each appeared only twice. To simplify the factor levels, we combined them into the "other" category.

In the community_district_num variable, some districts appeared only once or twice in the dataset. To ensure compatibility with the test set and avoid unseen factor levels during validation, we grouped these rare districts into a single "other" category.

In the zip_code column, we extracted values from the full_address_or_zip_code, URL, and url fields to fill in missing zip codes. Since the missing values were often present in one of the other columns, we used them to cross-reference and complete the data. There is an interesting about zip_code is that if we use as a numeric, the performance metric is a bit better than categorical.

Throughout the dataset, other textual inconsistencies such as typos (e.g., "yes89" in dogs_allowed, "efficiemcy" in kitchen_type) were cleaned using manual corrections and pattern collapsing.

The majority of the features have missingness, and we are going to use missForest to impute those features. However, before we impute, we need to do some preparation. There is a lot of missingness in the response variable sale_price, and only 528 values are not missing. So, we can only use those 528 data points for training and testing. That said, we can still use all of the data points for imputation purposes. Although we can include all rows in imputation, we cannot include y_select and y_test in the imputation step, because it could create correlations between the response and predictors. This would prevent us from getting a proper out-of-sample or honest performance metric. So, we are going to store y_select and y_test separately, set those values to NA in the dataset, and then run imputation.

3 Modeling

3.1 Regression Tree Modeling

We fit one regression tree model using the YARFCART function from the YARF package. One important hyperparameter in regression trees is the nodesize, which controls the minimum number of observations required in a leaf node. The default nodesize is 5. However, we are going to split into train, select and test to find the best nodesize.

To understand the predictive structure of the tree, we visualized the top layers using illustrate_trees() up to depth 6. This allowed us to inspect the most important early splits in the tree. The feature listing_price_to_nearest_1000 stood out as especially important, appearing most frequently in the first 6 layers of trees.

We also extracted the top 10 features used most frequently in the first six layers of the tree. These features are:

1. listing_price_to_nearest_1000
2. total_taxes
3. walk_score
4. general_cost

5. approx_year_built
6. sq_footage
7. parking_charges
8. community_district_num
9. fuel_type_gas
10. num_bedrooms

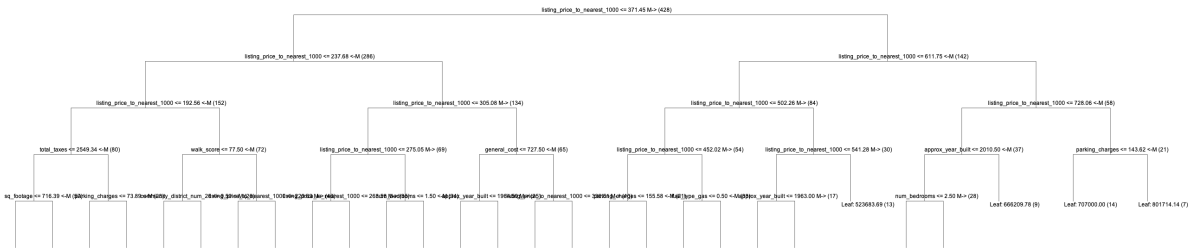

Figure 1: Regression Tree with 6 depths

3.2 Linear Modeling

We fit a linear model using the lm() function in R. Comparing with the most important feature found in the regression tree — listing_price_to_nearest_1000, we observe that its coefficient in the linear model is not the largest, but it is positive and reasonably large, confirming its strong relationship with sale_price.

In terms of performance, the linear model achieves an in-sample RMSE of 37356.34 and $R^2 = 0.9554128$, while the out-of-sample RMSE is 72428.53 with $R^2 = 0.8535482$. These results suggest that the model is not overfitting, as the gap between training and testing performance is not extreme. Likewise, the model is not underfitting, given that both R^2 values are relatively high. Since the out-of-sample performance remains strong, this indicates a fairly linear relationship between the features and the response. Therefore, we conclude that a linear model is a reasonable and effective choice for prediction in this context, supported by honest performance metrics.

Feature	**Coefficient Estimate**
(Intercept)	-473888.99
approx_year_built	196.72
cats_allowedyes	8374.38
community_district_num25	12593.04
community_district_num26	43658.51
community_district_num27	19753.01
community_district_num28	40048.01
community_district_num29	54724.21
community_district_num30	-2302.80
community_district_numother	6592.63
coop_condocondo	43420.97
dining_room_typeformal	758.51
dining_room_typeother	-10737.73
dogs_allowedyes	25.59
fuel_typegas	40350.29
fuel_typenone	83549.78
fuel_typeoil	35401.43
fuel_typeother	34193.84
garage_existsyes	8384.97
kitchen_typeeat_in	-3355.88
kitchen_typeefficiency	-7629.90
kitchen_typenone	35665.53
num_bedrooms	8834.85
num_floors_in_building	-560.08
num_full_bathrooms	3853.92
num_half_bathrooms	-20307.49
num_total_rooms	-744.52
parking_charges	-9.05
pct_tax_deductibl	1143.05
sq_footage	14.77
total_taxes	-7.07
walk_score	-376.27
listing_price_to_nearest_1000	803.62

Table 2: OLS Coefficient Estimates for Full Linear Model

Feature	**Coefficient Estimate**
general_cost	33.49
zip_code11005	37222.82
zip_code11101	45647.36
zip_code11102	110531.55
zip_code11104	-13916.73
zip_code11105	145567.72
zip_code11106	54677.04
zip_code11354	40945.85
zip_code11355	18712.01
zip_code11356	-18965.67
zip_code11357	14692.96
zip_code11358	25425.00
zip_code11360	20229.15
zip_code11361	14313.50
zip_code11362	-3468.49
zip_code11363	24184.85
zip_code11364	-19728.24
zip_code11365	-19334.07
zip_code11367	31669.53
zip_code11368	-8856.09
zip_code11369	32552.15
zip_code11370	44266.56
zip_code11372	65000.15
zip_code11373	35227.73
zip_code11374	20443.64
zip_code11375	16571.82
zip_code11377	45986.20
zip_code11378	45700.63
zip_code11379	54053.43
zip_code11385	27447.69

Table 3: OLS Coefficient Estimates for Full Linear Model

Feature	Coefficient Estimate
zip_code11414	-33604.31
zip_code11415	-7962.11
zip_code11417	-35778.22
zip_code11421	14518.13
zip_code11422	-67113.98
zip_code11423	-35747.91
zip_code11426	2395.70
zip_code11427	-29625.39
zip_code11432	-12512.25
zip_code11435	377.92
zip_codeother	-82447.58

Table 4: OLS Coefficient Estimates for Full Linear Model

3.3 Random Forest Modeling

This should be our choice of prediction model because, in terms of bias-variance decomposition, the bootstrap aggregation and the random selection of variables via mtry help reduce variance, while allowing a small nodesize (e.g., 1) keeps the bias low. However, mtry and the number of trees are hyperparameters that must be carefully tuned. The number of trees is relatively straightforward more trees generally reduce variance and stabilize the prediction. On the other hand, mtry needs to be tuned using a validation set or cross-validation; we used a 3-way split and selected the best mtry based on out-of-sample error.

Random forests are considered non-parametric models because they do not assume a fixed functional form; however, they are tunable via hyperparameters like mtry, nodesize, and ntree. By choosing this model, we gain the flexibility to manage variance and bias separately. One drawback in our specific case is that our dataset has relatively small n (sample size) and many features (around 20+), which can make the model prone to overfitting—especially if many features have high-cardinality levels or rare categories. The randomness introduced at each split can also become less effective when a few features dominate in importance, or when others carry little predictive value.

4 Performance Results

Model	**In-Sample RMSE**	**In-Sample R^2**	**OOS RMSE**	**OOS R^2**
OLS	37,356.34	0.9554	72,428.53	0.8535
Regression Tree	36,517.87	0.9574	85,425.41	0.7963
Random Forest	59,284.73	0.8877	102,175.30	0.7085

Table 5: In-Sample and Out-of-Sample RMSE and R^2 for OLS, Regression Tree, and Random Forest Models

For OLS, we can clearly see that the data has some linearity, which makes it the best model out of the three when looking at out-of-sample performance. The OLS model achieves the lowest OOS RMSE and the highest R^2, which suggests that it generalizes better than the others.

The regression tree is our second-best model based on both in-sample and out-of-sample metrics. It captures non-linear structure better than OLS but slightly overfits, as seen from the drop in R^2 when moving to test data.

Random forest, which is typically expected to perform well, ended up being the worst performer in our case. As we mentioned before, this is likely due to a few reasons: the small number of training examples, the large number of features, and the added randomness from hyperparameters like mtry. These factors combined may have caused instability and overfitting in the random forest model. If we had more data, we believe that random forest would eventually outperform the others due to its ability to capture complex interactions.

When thinking about extrapolation, OLS has an advantage because it uses a global linear form, so it's smoother and more stable outside the training domain. The tree models (especially regression tree and random forest), on the other hand, are based on recursive partitioning and may not handle values outside of the training set well. That's another reason OLS works better here, but this could change as data grows.

The out-of-sample estimates we calculated are honest and valid indicators of future performance. We created a separate test set and never used it during model training or tuning. So the OOS metrics realistically reflect how well each model would perform when making predictions on truly unseen data.

5 Discussion

To summarize, we've seen some interesting things throughout building this model. One surprising result was that using zip code as a numeric variable gave us better performance than using it as a categorical variable. Another finding was that using the default 2-way split in random forest worked better than when we used a 3-way split. That said, we had to drop about 1500 rows due to missingness in the response variable, which definitely hurt our training size. If we had more training data, we believe that the random forest model would likely outperform the others.

The overall model performance really came down to how we handled the data and the cleaning process. Speaking of the 3-way split, we made sure to lock y_select and y_test before imputation, which gave us honest out-of-sample metrics. It's also interesting to see that OLS ended up beating both tree-based models, which we didn't expect at the beginning.

One reason this might have happened is due to missForest. Since it randomly imputes missing values each time, we noticed that our performance metrics changed slightly depending on the run. So, there is some randomness even before modeling begins.

We don't think this model is production ready yet. There are still things we could tune better, and more importantly, we need more high-quality training data to fully unlock the potential of models like random forest. And no, we don't think we can beat Zillow right now—the OOS metrics just aren't strong enough yet.

Code Appendix

````
---
title: "Final_Project"
Name: Ye Htut Maung
date: "2025-05-25"
---

# Packages/Libraries Setup

```{r}
pacman::p_load(rJava, readr, stringr, pander, checkmate, ggplot2, gridExtra, R6, Bolsta
if (!pacman::p_isinstalled(YARF)){
  pacman::p_install_gh("kapelner/YARF/YARFJARs", ref = "dev")
  pacman::p_install_gh("kapelner/YARF/YARF", ref = "dev", force = TRUE)
}
pacman::p_load(YARF)
options(java.parameters = "-Xmx8000m")
pacman::p_load(data.table, tidyverse, magrittr, ggplot2, skimr, lubridate, missForest, 
pacman::p_load(mlr3, mlr3learners, mlr3tuning, mlr3verse, paradox)
```



# Data Import and skimming data

```{r}
Xy = fread("housing_data_2016_2017.csv")
skim(Xy)
head(Xy, 5)
nrow(Xy)
````
````

```
ncol(Xy)
```

```
# Data Preparation
# dropping irrelevent features
```{r}
# List of columns to drop
cols_to_drop = c(
  "HITId", "HITTypeId", "Title", "Description", "Reward", "CreationTime",
  "RequesterAnnotation", "Expiration", "AssignmentId", "WorkerId",
  "AssignmentStatus", "AcceptTime", "SubmitTime", "AutoApprovalTime",
  "ApprovalTime", "LifetimeApprovalRate", "Last30DaysApprovalRate", "Last7DaysApprovalRa
  "Keywords", "NumberOfSimilarHITs", "LifetimeInSeconds", "RejectionTime", "RequesterFee
  "MaxAssignments", "AssignmentDurationInSeconds", "AutoApprovalDelayInSeconds", "WorkTi
  "model_type", "date_of_sale"
) # I drop URL as I can get zipcode from full_address_or_zip_code

# Drop the columns
Xy_new =  Xy[, !names(Xy) %in% cols_to_drop, with = FALSE]
skim(Xy_new)
head(Xy_new, 5)
ncol(Xy_new)
```

We left with 27 featuers.

```
# cat allowed
``` {r}
# cats_allowed (yes, no)
Xy_new[["cats_allowed"]]
sum(is.na(Xy_new$cats_allowed)) # no missing
Xy_new[cats_allowed == "y", cats_allowed := "yes"]
Xy_new[, cats_allowed := as.factor(cats_allowed)]

Xy_new[["cats_allowed"]]
summary(Xy_new$cats_allowed)
```

```
# common_charges and maintenance_cost
```

```` {r}
# common_charges is for condo
Xy_new[["common_charges"]]
# maintenance_cost is for co op
Xy_new[["maintenance_cost"]]
# combine common_charges and maintenance_cost into one column and called it general_cost
Xy_new[, general_cost := fifelse(!is.na(common_charges), common_charges, maintenance_cos
table(Xy_new$general_cost)
#checking
Xy_new[, .(common_charges, maintenance_cost, general_cost)]
# drop the common_charges and maintenance_cost
Xy_new[, c("common_charges", "maintenance_cost") := NULL]
sum(is.na(Xy_new$general_cost)) # 100 missing
# remove $ and , and conver to numeric
Xy_new[, general_cost := as.numeric(gsub("[\\$,]", "", general_cost))]
summary(Xy_new$general_cost)
sd(Xy_new$general_cost, na.rm = TRUE)
```


*# coop_condo*
```` {r}

Xy_new[["coop_condo"]]
unique(Xy_new$coop_condo)
sum(is.na(Xy_new$coop_condo)) # no missing

Xy_new[, coop_condo := as.factor(coop_condo)]
table(Xy_new$coop_condo)
```

*# dining_room_type (make it factor)*
````{r}
table(Xy_new$dining_room_type)
Xy_new[dining_room_type %in% c("dining area", "none"), dining_room_type := "other"]
table(Xy_new$dining_room_type)
Xy_new[["dining_room_type"]]
Xy_new[, dining_room_type := as.factor(dining_room_type)]
levels(Xy_new$dining_room_type)
sum(is.na(Xy_new$dining_room_type))
```

*# dogs_allowed*

```{r}
table(Xy_new$dogs_allowed)
Xy_new[dogs_allowed == "yes89", dogs_allowed := "yes"]
Xy_new[, dogs_allowed := as.factor(dogs_allowed)]
table(Xy_new$dogs_allowed)
sum(is.na(Xy_new$dogs_allowed))
```

*# fuel_type*
```{r}
# fuel_type
table(Xy_new$fuel_type)
# Combine "Other" into "other"
Xy_new[fuel_type %in% c("Other"), fuel_type := "other"]
# Convert to factor
Xy_new[, fuel_type := as.factor(fuel_type)]
table(Xy_new$fuel_type)
levels(Xy_new$fuel_type)
sum(is.na(Xy_new$fuel_type))
```

*# garage_exists*
```{r}

table(Xy_new$garage_exists)
Xy_new[["garage_exists"]]
# Normalize values to lowercase
Xy_new[, garage_exists := tolower(garage_exists)]
# Set valid yes values
Xy_new[garage_exists %in% c("1", "eys", "ug", "underground"), garage_exists := "yes"]
# Create binary indicator: 1 if garage exists, 0 otherwise
Xy_new[is.na(garage_exists), garage_exists := "no"]
Xy_new[, garage_exists := as.factor(garage_exists)]
sum(is.na(Xy_new$garage_exist))
table(Xy_new$garage_exists)
```

*# kitchen_type*
```{r}
table(Xy_new$kitchen_type)
# Make everything lowercase
Xy_new[, kitchen_type := tolower(kitchen_type)]
# Collapse "eat in" variations into "eat_in"
Xy_new[kitchen_type %in% c("eat in", "eatin"), kitchen_type := "eat_in"]
```

```
# Collapse "efficiency" variations into "efficiency"
Xy_new[kitchen_type %in% c("efficiemcy", "efficiency kitchen", "efficiency kitchene",
                           "efficiency ktchen", "efficiency kitchene"),
       kitchen_type := "efficiency"]
Xy_new[kitchen_type %in% "1955", kitchen_type := "none"]
# Convert to factor
Xy_new[, kitchen_type := as.factor(kitchen_type)]
# View cleaned result
table(Xy_new$kitchen_type)
sum(is.na(Xy_new$kitchen_type))
```

*# parking_charges*

```{r}
table(Xy_new$parking_charges)
# Remove $ and convert to numeric
Xy_new[, parking_charges := as.numeric(gsub("\\$", "", parking_charges))]
summary(Xy_new$parking_charges)
sum(is.na(Xy_new$parking_charges))
sd(Xy_new$parking_charges, na.rm=TRUE)
```

*# sale_price*

```{r}
table(Xy_new$sale_price)
Xy_new[["sale_price"]]
# Remove dollar sign and convert to numeric
Xy_new[, sale_price := as.numeric(gsub("[\\$,]", "", sale_price))]
summary(Xy_new$sale_price)
```

*# total_taxes*

```{r}
table(Xy_new$total_taxes)
Xy_new[, total_taxes := as.numeric(gsub("[\\$,]", "", total_taxes))]
summary(Xy_new$total_taxes)
sd(Xy_new$total_taxes, na.rm=TRUE)
```

*# listing_price_to_nearest_1000*

```{r}
table(Xy_new$listing_price_to_nearest_1000 )
```

```
Xy_new[["listing_price_to_nearest_1000"]]
Xy_new[, listing_price_to_nearest_1000 := as.numeric(gsub("[\\$,]", "", listing_price_to
summary(Xy_new$listing_price_to_nearest_1000)
sd(Xy_new$listing_price_to_nearest_1000, na.rm=TRUE)
```

```{r}
# approx_year_built
Xy_new[["approx_year_built"]]
summary(Xy_new$approx_year_built)
sd(Xy_new$approx_year_built, na.rm=TRUE)
```

*# community_district_num*
```{r}
Xy_new[["community_district_num"]]
table(Xy_new$community_district_num)
Xy_new[, community_district_num := as.character(community_district_num)]
# Define the values to collapse into "other"
rare_districts = c(3, 4, 6, 8, 11, 12, 13, 14, 15, 16, 18, 19, 20, 22, 23, 31, 32)

# Convert those values to "other"
Xy_new[community_district_num %in% rare_districts, community_district_num := "other"]

# Finally, convert to factor
Xy_new[, community_district_num := as.factor(community_district_num)]

table(Xy_new$community_district_num)
summary(Xy_new$community_district_num)
sum(is.na(Xy_new$community_district_num))
```

*# num_bedrooms*
```{r}
Xy_new[["num_bedrooms"]]
summary(Xy_new$num_bedrooms)
sd(Xy_new$num_bedrooms, na.rm=TRUE)
```

*# num_floors_in_building*

```{r}
Xy_new[["num_floors_in_building"]]
table(Xy_new$num_floors_in_building)
summary(Xy_new$num_floors_in_building)
sd(Xy_new$num_floors_in_building, na.rm=TRUE)
```

*# num_full_bathrooms*
``` {r}
Xy_new[["num_full_bathrooms"]]
table(Xy_new$num_full_bathrooms)
summary(Xy_new$num_full_bathrooms)
sd(Xy_new$num_full_bathrooms, na.rm=TRUE)
sum(is.na(Xy_new$num_full_bathrooms))
```

*# num_half_bathrooms*
```{r}
Xy_new[["num_half_bathrooms"]]
table(Xy_new$num_half_bathrooms)
summary(Xy_new$num_half_bathrooms)
sd(Xy_new$num_half_bathrooms, na.rm=TRUE)
sum(is.na(Xy_new$num_half_bathrooms))
```

*# num_total_rooms*
```{r}
Xy_new[["num_total_rooms"]]
table(Xy_new$num_total_rooms)
summary(Xy_new$num_total_rooms)
sd(Xy_new$num_total_rooms, na.rm=TRUE)
```

*# pct_tax_deductibl*
```{r}
Xy_new[["pct_tax_deductibl"]]
table(Xy_new$pct_tax_deductibl)
summary(Xy_new$pct_tax_deductibl)
sd(Xy_new$pct_tax_deductibl, na.rm=TRUE)
```

*# sq_footage*

```{r}
Xy_new[["sq_footage"]]
table(Xy_new$sq_footage)
summary(Xy_new$sq_footage)
sd(Xy_new$sq_footage, na.rm=TRUE)
```


*# walk_score*
```{r}
Xy_new[["walk_score"]]
table(Xy_new$walk_score)
summary(Xy_new$walk_score)
sd(Xy_new$walk_score, na.rm=TRUE)
```


*# full_address_or_zip_code and URL*
```{r}

Xy_new[, zip_code := stringr::str_extract(full_address_or_zip_code, "\\d{5}$")]
Xy_new[, "full_address_or_zip_code" := NULL]
# Xy_new[, zip_code := as.factor(zip_code)]


# Create a new column 'final_url' taking non-missing values from URL or url
Xy_new[, final_url := fifelse(!is.na(URL), URL, url)]
Xy_new[, c("URL", "url") := NULL]
head(Xy_new$final_url, 5)
Xy_new[, zip_from_url := stringr::str_extract(final_url, "(?<=-)[0-9]{5}(?=-[0-9]+$)")]

# Xy_new[, zip_from_url := as.factor(zip_from_url)]
# Fill missing zip_code using zip_from_url
Xy_new[is.na(zip_code), zip_code := zip_from_url]
# Drop the temporary zip_from_url column and final_url
Xy_new[,c("final_url", "zip_from_url") := NULL]
table(Xy_new$zip_code)
sum(is.na(Xy_new$zip_code))
# making other for the zip code that is less than 3
# Count frequency of each zip code
zip_counts = table(Xy_new$zip_code)
zip_counts
# Identify zip codes with < 3 instances
rare_zips = names(zip_counts[zip_counts < 3])
```

```
rare_zips
# Collapse rare zip codes into "other"
Xy_new[zip_code %in% rare_zips, zip_code := "other"]

# Convert to factor
Xy_new[, zip_code := as.factor(zip_code)]

table(Xy_new$zip_code)
summary(Xy_new$zip_code)
#sd(Xy_new$zip_code, na.rm=TRUE)
```

```{r}
ncol(Xy_new)
```



*# Data Cleaning I (Errors, Duplicates, Outliers Handling)*

*# Data Manipulation (Featurization, Imputation)*
```{r}
head(Xy_new, 900)
# ensures non-missing come first (FALSE < TRUE)
Xy_new = Xy_new[order(is.na(sale_price))]
sum(is.na(Xy_new$sale_price))
which(!is.na(Xy_new$sale_price))
last_non_missing_y_index = max(range(which(!is.na(Xy_new$sale_price))))
last_non_missing_y_index
```

*# Imputation*

```{r}
# Ensure it's a data.frame
is.data.frame(Xy_new)

# Define total number of non-missing y rows
total_non_missing = last_non_missing_y_index

# Set sizes explicitly
select_size = 100
test_size = 100
```

```
# Get index ranges
select_indices = 1:select_size
test_indices = (select_size + 1):(select_size + test_size)
train_indices = (select_size + test_size + 1):total_non_missing

# Subset the data
D_select = Xy_new[select_indices]
D_test = Xy_new[test_indices]
D_train = Xy_new[train_indices]

# Extract y values
y_select = D_select$sale_price
y_test = D_test$sale_price
```

*# Pre-imputation Locking out the y_test*

```{r}
# Start from a copy of Xy_new (already sorted so rows 1:528 have non-missing y)
Xy_pre_impute = copy(Xy_new)

# Combine indices for test and select sets
holdout_indices = c(test_indices, select_indices)

# Mask sale_price values in both sets
Xy_pre_impute[holdout_indices, sale_price := NA]
```

*# imputation time*

```{r}
# Apply missForest
Xy_imputed = missForest(Xy_pre_impute)$ximp
Xy_imputed
```

*# Cleaning after imputation (round numbers that can't have decimals)*

```{r}
cols_to_round = c(
  "num_bedrooms",
  "num_floors_in_building",
  "num_half_bathrooms",
  "num_total_rooms",
  "approx_year_built"
)
```

```
# Round and convert to integer
Xy_imputed[, (cols_to_round) := lapply(.SD, function(x) as.integer(round(x))), .SDcols =
Xy_imputed
```


*# spliting into X_train, X_test, y_train, y_test*

```{r}
# Training set
X_train = Xy_imputed[train_indices, !"sale_price", with = FALSE]
y_train = Xy_imputed[train_indices, sale_price]

# Test set
X_test = Xy_imputed[test_indices, !"sale_price", with = FALSE]
# y_test was saved before imputation and should not be overwritten

# Select (validation) set
X_select = Xy_imputed[select_indices, !"sale_price", with = FALSE]
# y_select was also saved before imputation and should remain untouched
```



*# Regression Tree Modeling*
Finding the best nodesize
```{r}

# Set max nodesize to test
max_nodesize_to_try = 70

# Initialize vectors to store errors
in_sample_errors = numeric(max_nodesize_to_try)
oos_errors = numeric(max_nodesize_to_try)

# Loop through possible nodesize values
for (i in 1:max_nodesize_to_try) {
  tree_mod = YARFCART(data.frame(X_train), y_train, nodesize = i, calculate_oob_error =

  # In-sample prediction
  yhat_train = predict(tree_mod, data.frame(X_train))
  in_sample_errors[i] = sd(y_train - yhat_train)

  # Out-of-sample (validation) prediction
```

```
  yhat_select = predict(tree_mod, data.frame(X_select))
  oos_errors[i] = sd(y_select - yhat_select)
}


ggplot(data.frame(
  nodesize = 1:max_nodesize_to_try,
  in_sample_errors = in_sample_errors,
  oos_errors = oos_errors
)) +
  geom_point(aes(x = nodesize, y = in_sample_errors), col = "red") +
  geom_line(aes(x = nodesize, y = in_sample_errors), col = "red") +
  geom_point(aes(x = nodesize, y = oos_errors), col = "blue") +
  geom_line(aes(x = nodesize, y = oos_errors), col = "blue") +
  labs(
    title = "In-Sample and OOS Error vs Nodesize",
    x = "nodesize",
    y = "Standard Deviation Error"
  ) +
  theme_minimal()
```

```{r}
best_nodesize = which.min(oos_errors)
cat("Best nodesize based on validation set:", best_nodesize, "\n")
```

```{r}

X_train_select = rbind(X_train, X_select)
y_train_select = c(y_train, y_select)
tree_mod = YARFCART(X_train_select,y_train_select, nodesize = best_nodesize, calculate_o
```
# illustrate the tree
```{r}
get_tree_num_nodes_leaves_max_depths(tree_mod)
illustrate_trees(tree_mod, max_depth = 6, open_file = TRUE)

```

```
# prediction on Regression tree
# in-sample
```{r}

y_hat_train_select = predict(tree_mod, X_train_select)
rmse_train_select = sqrt(mean((y_train_select - y_hat_train_select)^2))
ss_res_train_select = sum((y_train_select - y_hat_train_select)^2)
ss_tot_train_select = sum((y_train_select - mean(y_train_select))^2)
r2_train_select = 1 - ss_res_train_select / ss_tot_train_select
rmse_train_select
r2_train_select
```



# out-of-sample
```{r}
y_hat_test = predict(tree_mod, X_test)
oos_rmse_tree = sqrt(mean((y_test - y_hat_test)^2))
ss_res = sum((y_test - y_hat_test)^2)
ss_tot = sum((y_test - mean(y_test))^2)
oos_r2_tree = 1 - ss_res / ss_tot
oos_rmse_tree
oos_r2_tree
```



# Linear Modeling

```{r}
lm_model = lm(y_train_select ~ ., data = data.frame(X_train_select, y_train_select))
round(coef(lm_model), 3)

```

# in sample residual
```{r}
y_hat_lm_train_select = predict(lm_model,X_train_select)
rmse_lm_train_select = sqrt(mean((y_train_select - y_hat_lm_train_select)^2))
rmse_lm_train_select

ss_res = sum((y_train_select - y_hat_lm_train_select)^2)
ss_tot = sum((y_train_select - mean(y_train_select))^2)
```
```

```
r2_lm_train_select = 1 - ss_res / ss_tot
r2_lm_train_select
```

# oos
```{r}
oos_y_hat_lm_test = predict(lm_model, X_test)
oos_rmse_lm_test = sqrt(mean((y_test - oos_y_hat_lm_test)^2))
oos_rmse_lm_test
ss_res = sum((y_test - oos_y_hat_lm_test)^2)
ss_tot = sum((y_test - mean(y_test))^2)

r2_lm_test = 1 - ss_res / ss_tot
r2_lm_test
```

# Random Forest Modeling

```{r}
library(randomForest)
# Combine X_train and y_train into a single data frame
df_train = data.frame(X_train)
df_train$sale_price = y_train

df_select = data.frame(X_select)
df_select$sale_price = y_select

p = ncol(X_train)  # number of predictors
num_trees = 1200
oos_se_by_mtry = numeric(p)

# Loop over mtry from 1 to p
for (m in 1:p) {
  rf_model = randomForest(sale_price ~ ., data = df_train, ntree = num_trees, mtry = m)
  y_hat_select = predict(rf_model, newdata = df_select)
  oos_se_by_mtry[m] = sqrt(mean((df_select$sale_price - y_hat_select)^2))  # RMSE
}

# Create data frame for plotting
df_mtry = data.frame(
  mtry = 1:p,
```

````
  oos_rmse = oos_se_by_mtry
)

# Plot OOS RMSE vs mtry
library(ggplot2)
ggplot(df_mtry, aes(x = mtry, y = oos_rmse)) +
  geom_point() +
  geom_line() +
  labs(
    title = "OOS RMSE vs mtry (500 Trees)",
    x = "mtry (Number of Features Tried at Each Split)",
    y = "Out-of-Sample RMSE on Select Set"
) +
  theme_minimal()

# Find best mtry and fit final model
best_mtry = which.min(oos_se_by_mtry)
cat("Best mtry based on OOS RMSE:", best_mtry, "\n")

```



```{r}
seed = 42
num_trees = 1000
mod_rf = YARF(X_train_select, y_train_select, num_trees = num_trees, seed = seed, mtry =
mod_rf
```
in-sample
```{r}

y_hat_rf_train_select = predict(mod_rf, X_train_select)

rmse_rf_train_select = sqrt(mean((y_train_select - y_hat_rf_train_select)^2))

ss_res_train_select = sum((y_train_select - y_hat_rf_train_select)^2)
ss_tot_train_select = sum((y_train_select - mean(y_train_select))^2)
r2_rf_train_select = 1 - ss_res_train_select / ss_tot_train_select

cat("In-sample RMSE:", rmse_rf_train_select, "\n")
cat("In-sample R²:", r2_rf_train_select, "\n")
```
````
````

```
# out-of-sample
```{r}

y_hat_rf_test = predict(mod_rf, X_test)
rmse_rf_test = sqrt(mean((y_test - y_hat_rf_test)^2))
ss_res_test = sum((y_test - y_hat_rf_test)^2)
ss_tot_test = sum((y_test - mean(y_test))^2)
r2_rf_test = 1 - ss_res_test / ss_tot_test
cat("Out-of-sample RMSE:", rmse_rf_test, "\n")
cat("Out-of-sample R²:", r2_rf_test, "\n")
```


Shipping Final Model Trained On All Data
```{r}
# Combine X and y from train, select, and test
X_final = rbind(X_train, X_select, X_test)
y_final = c(y_train, y_select, y_test)

# Fit final YARF model
mod_rf_final = YARF(X_final, y_final,
                    num_trees = num_trees,
                    seed = seed,
                    mtry = best_mtry,
                    nodesize = 1,   calculate_oob_error = FALSE, verbose = FALSE)
mod_rf_final
```


```{r}

```
```